

Algorithmic Structure of the Stabilized Porous Navier–Stokes Solver

Antigravity AI Project Authors

May 17, 2026

Abstract

This document specifies the complete algorithmic pipeline that drives a single invocation of `run_simulation` in the stabilized porous Navier–Stokes solver. The pipeline is presented as a cascade of seven nested algorithms—one outermost orchestration box, three for the inner Newton machinery, one for the Picard globalization fallback, one for the staggered Orthogonal Sub-Grid Scale (OSGS) outer loop, and one for the Method of Manufactured Solutions (MMS) plateau verifier. For each algorithm we provide both rigorous pseudocode and pedagogical exposition: what the algorithm does, why each branch exists, how its numerical parameters influence cost and robustness, and how it couples to its neighbours in the cascade. A separate section documents three adaptations that live exclusively in the MMS sweep harness and do *not* participate in single-shot `run_simulation` invocations.

Contents

1	Overview and Algorithmic Challenges	2
2	Parameter Taxonomy and Code Equivalence	3
2.1	Numerical parameters	3
2.2	Algorithm–to–code correspondence	3
3	The Outermost Orchestration: Algorithm 1	3
4	The Inner Newton Machinery	4
4.1	The merit function and its diagonal preconditioning	4
4.2	Algorithm A: the safeguarded Newton outer loop	5
4.3	Algorithm A.1: linear-system solve and NaN guard	6
4.4	Algorithm A.2: Armijo backtracking line search	6
4.5	Algorithm A.3: termination safeguards	7
5	The Picard Fallback	8
6	OSGS Staggered Relaxation	9
6.1	Algorithm C	10
6.2	The bootstrap and why the ASGS state matters	10
6.3	Why projection happens on the unconstrained spaces	11
6.4	Mass-matrix pre-factorizing	11
6.5	Warmup phase	11
6.6	State drift: ℓ^∞ vs continuous L^2	11
6.7	Projection drift, in the mass-weighted inner product	11
6.8	Choosing the stopping mode	12
6.9	Block-wise Anderson acceleration	12
6.10	The reaction-projection trim	12
6.11	Failure of the inner Newton	13

7	MMS Plateau Verification	13
7.1	Numerical convergence versus discretisation convergence	13
7.2	Algorithm D	13
7.3	Three norms, three floors	13
7.4	Consecutive passes and budget exhaustion	14
7.5	Two entry points and how they differ	14
8	The MMS Sweep Harness	14
8.1	Adaptive τ_{ftol}	14
8.2	Adaptive noise floor	15
8.3	Per-case Picard widening	15
8.4	Closing remark	15
9	Convergence-Analysis Notes	15
9.1	The false-equivalency failsafe	15
9.2	Why τ_{stag} must scale with conditioning	16
9.3	Why the OSGS stopping mode matters	16
A	Code–Notation Correspondence	16

1 Overview and Algorithmic Challenges

The continuous problem is the stationary, incompressible, generalized porous Navier–Stokes system on a domain $\Omega \subset \mathbb{R}^d$,

$$\begin{aligned} \mathbf{u} \cdot \nabla \mathbf{u} - \nabla \cdot (2\nu \nabla^s \mathbf{u}) + \sigma(\alpha, \mathbf{u})\mathbf{u} + \nabla p &= \mathbf{f}, \\ \nabla \cdot \mathbf{u} &= 0, \end{aligned} \tag{1}$$

discretised by an equal-order Lagrangian pair $(\mathbf{V}_h, Q_h)$ of degree $(k_v, k_p) = (k, k)$ and stabilised by a Variational Multiscale (VMS) formulation. The drag tensor $\sigma(\alpha, \mathbf{u}) = a(\alpha) + b(\alpha)|\mathbf{u}|$ couples the Darcy–Forchheimer regime to the convective momentum balance; the porosity field α varies in space, and the high-Re, high-Da regimes the solver must handle are exactly those in which the system becomes simultaneously convection-dominated, reaction-dominated, and stiff.

Two algorithmic challenges follow.

Nonlinearity from convection and Forchheimer drag. Newton’s method converges quadratically once the iterate enters its quadratic basin, but locating that basin from an arbitrary initial guess is non-trivial when both $\mathbf{u} \cdot \nabla \mathbf{u}$ and the $|\mathbf{u}|$ factor in σ amplify any error in $\mathbf{u}$. Outside the basin, a raw Newton step typically over-corrects, and the merit function $\Phi(\mathbf{x}) = \frac{1}{2}\|\mathbf{b}(\mathbf{x})\|^2$ grows. The solver must detect this divergence and respond—either by shortening the step (line search), by switching the iteration map to one with a wider basin of attraction (Picard), or by aborting cleanly.

Implicit OSGS projection. In the Orthogonal Sub-Grid Scale formulation the sub-grid component lives in the orthogonal complement $\mathcal{X}_{h0}^\perp$ of the finite element space; in practice the projection $\boldsymbol{\pi}_h \in \mathcal{X}_h$ of the strong residual is computed, and the sub-grid scale is identified with $(I - \Pi_h)\mathcal{R}$. The projection itself depends on the primary fields $U_h = (\mathbf{u}_h, p_h)$, so a fully monolithic Jacobian would embed the Fréchet derivative of an L^2 projection operator. Rather than carrying this dense block, the solver freezes $\boldsymbol{\pi}_h$, solves the nonlinear primary system, then re-projects, in a staggered (block-coordinate) fixed point.

Architecture of the response. The architecture that emerges is a two-stage cascade. *Stage I* drives the algebraic system to a converged ASGS state, regardless of the final stabilization method requested. It composes an exact-Newton attempt, a Picard fallback, and a final Newton polish (Algorithms 2 and 6). *Stage II* either reports the ASGS state directly or, when the user has selected OSGS, runs a staggered outer loop on $(U_h, \boldsymbol{\pi}_h)$ initialised at the Stage I solution (Algorithm 7). An optional MMS verifier (Algorithm 8) extends each Stage II loop with a plateau test that distinguishes *algebraic* convergence (small nonlinear residual) from *discretisation* convergence (small error against an exact solution). The outermost orchestration (Algorithm 1) wires all of these together. The reader who wants a single mental model should hold:

ASGS is always run first; OSGS, when requested, refines the ASGS state by a staggered projection loop; MMS verification, when enabled, runs as a plateau gate sitting at the top of whichever Stage II loop is active.

2 Parameter Taxonomy and Code Equivalence

2.1 Numerical parameters

Table 1 enumerates every numerical parameter consumed by the algorithms below, grouped by role. Parameters marked *harness-only* live in the configuration schema but are read *only* by the MMS sweep harness (`test/extended/ManufacturedSolutions/run.test.jl`); see Section 8. All remaining parameters participate in a single `run.simulation` call.

2.2 Algorithm-to-code correspondence

Table 2 pins each algorithm presented below to its Julia entry point.

3 The Outermost Orchestration: Algorithm 1

The orchestration drives one simulation end-to-end. Its central organising principle is the two-stage cascade described in Section 1: an unconditional ASGS algebraic initialisation followed by a method-dependent dispatch. Notice that Stage I runs even when the user has requested OSGS, because the staggered projection loop is much better conditioned when initialised at a converged ASGS state than at a zero initial guess.

Algorithm 1: SimulationOrchestration: end-to-end driver.

Input: Configuration path P .

Output: (u_h, p_h, π_h) , success flag S , total inner-iteration count.

```

 $\Theta \leftarrow \text{load\_config\_with\_defaults}(P)$  ; // validate; reject unknown keys
 $M \leftarrow \text{build\_default\_mesh}(\Theta)$ ;
 $(X, Y, k_v, k_p) \leftarrow \text{build\_fe\_spaces}(M, \Theta)$  ; // Dirichlet-constrained trial/test pairs
 $(V_{\text{free}}, Q_{\text{free}}) \leftarrow \text{unconstrained mirrors of } (X, Y)$  ; // projection spaces
 $F \leftarrow \text{build\_formulation}(\Theta)$  ; // viscous + reaction +  $\tau$  + projection policy
 $T \leftarrow \text{FETopology}(X, Y, M, \Omega, d\Omega, V_{\text{free}}, Q_{\text{free}}, h, \mathbf{f}, \alpha)$ ;
 $\mathbf{x}^0 \leftarrow \text{interpolate\_everywhere}(0, X)$ ;

/* Stage I. Algebraic initialisation in ASGS mode ( $\pi_h \equiv 0$ ). */
 $(S, U_h^{\text{ASGS}}) \leftarrow \text{PicardFallback}(\mathbf{x}^0, \mathbf{x}_{\text{backup}}^0)$  ; // Alg. 6
if not  $S$  then
    | return failure (no fields written);
end

/* Stage II. Method dispatch. */
if  $\Theta.\text{method} = \text{ASGS}$  then
    | if  $\Theta.\text{mms.enabled}$  then
    | | run  $K_{\text{extra}}$  extension cycles, calling VerifyMMSPlateau each cycle (Alg. 8, ASGS entry);
    | end
    | return  $(U_h^{\text{ASGS}}, \pi_h \equiv 0, \dots)$ ;
end
if  $\Theta.\text{method} = \text{OSGS}$  then
    | pre-assemble and LU-factor  $M_u = (V_{\text{free}}, U_{\text{free}})_{L^2}$ ,  $M_p = (Q_{\text{free}}, P_{\text{free}})_{L^2}$  once;
    |  $\pi_h^0 \leftarrow L^2$ -projection of  $\mathcal{R}(U_h^{\text{ASGS}})$  onto  $(V_{\text{free}}, Q_{\text{free}})$ ;
    | run OSGSFractionalRelaxation( $U_h^{\text{ASGS}}, \pi_h^0$ ) (Alg. 7) ; // embeds Alg. 8 as plateau gate
end
export_results(VTK | HDF5);

```

Step by step. Lines 1–7 load and validate the configuration, build the mesh, and assemble the finite-element pair. The unconstrained spaces $V_{\text{free}}, Q_{\text{free}}$ are required *only* by the OSGS projection step (Section 6); they carry no Dirichlet data, because projecting onto a Dirichlet-constrained space introduces an $O(1)$ boundary residual that destroys the expected $O(h^{k+1})$ MMS rate. The `FETopology` bundle simply ties together the geometry, the test/trial spaces, the per-element mesh size h , and the loading data $(\mathbf{f}, \alpha)$.

Stage I (line 8) is the workhorse: it calls `PicardFallback` (Algorithm 6), which composes an exact-Newton attempt, a Picard globalisation pass, and a final Newton polish, all with $\pi_h \equiv 0$. The output U_h^{ASGS} is the algebraic root of the ASGS system; when Stage II is OSGS, the staggered loop will descend from this state with an already-converged primary field.

The Stage II dispatch (lines 11–19) is intentionally simple: either accept the ASGS state, optionally extended by an MMS plateau check, or hand off to the OSGS outer loop. The mass matrices M_u, M_p are pre-assembled and LU-factored exactly once before the OSGS loop begins (line 16). Because the projection target spaces $V_{\text{free}}, Q_{\text{free}}$ never change during the loop, these factorisations are reused at every outer iteration, which converts every L^2 projection into a constant-cost forward/back substitution.

Failure propagation. If Stage I returns $S = \text{False}$, the orchestrator returns immediately without writing output (lines 9–10). The diagnostics cache carries the algebraic exit reason so the calling script can distinguish a divergent run from a converged one.

Anderson and MMS coupling. Two side-effects of Θ deserve mention. First, if the configured accelerator type is "Anderson", the OSGS loop constructs a pair of block-wise Anderson accelerators (Section 6.9); the orchestrator itself does not allocate them, but their presence changes the iterate produced at each outer step. Second, when `Θ.mms.enabled`, the OSGS outer-iteration budget N_{OSGS} is silently inflated by K_{extra} so that the plateau verifier has room to run after the staggered fixed point itself has converged. Without this inflation, the verifier would chronically exhaust the budget on the very first extension cycle.

4 The Inner Newton Machinery

4.1 The merit function and its diagonal preconditioning

Let $\mathbf{b}(\mathbf{x}) \in \mathbb{R}^N$ be the assembled residual of the (possibly stabilised) discrete weak form, and $\mathcal{J}(\mathbf{x}) = \partial \mathbf{b} / \partial \mathbf{x}$ its Jacobian. The merit function used by the line search is the diagonally equilibrated quadratic

$$\Phi_{\mathbf{w}}(\mathbf{x}) = \frac{1}{2} \sum_{k=1}^N \left(\frac{b_k(\mathbf{x})}{w_k} \right)^2, \quad w_k = \max(|\mathcal{J}_{kk}|, \varepsilon_{\text{mach}} \|\text{diag}(\mathcal{J})\|_{\infty}, \varepsilon_{\text{mach}}). \quad (2)$$

The weights w_k are recomputed at every Newton iteration from the diagonal of the current Jacobian.

Why equilibrate? The discrete residual mixes momentum-balance rows—measured in physical units of force per unit volume, and possibly amplified by a large Reynolds or Damköhler number—with mass-balance rows, whose natural magnitude is dimensionally different. A raw unweighted merit $\frac{1}{2} \|\mathbf{b}\|^2$ would let the momentum rows dominate entirely, so the line search would be quietly indifferent to mass-conservation errors. Equilibrating by w_k rescales every row to its own natural unit, and the line search now respects both balances simultaneously. The lower bound $\varepsilon_{\text{mach}} \|\text{diag}(\mathcal{J})\|_{\infty}$ prevents a genuinely small diagonal entry (e.g. a pressure row when the system is saddle-point-like) from triggering division by zero.

An algebraic identity that simplifies the line search. For a Newton step $\delta \mathbf{x}$ satisfying $\mathcal{J} \delta \mathbf{x} = \mathbf{b}$, the directional derivative of $\Phi_{\mathbf{w}}$ along $-\delta \mathbf{x}$ at $\mathbf{x}$ is

$$D = - \sum_k w_k^{-2} b_k (\mathcal{J} \delta \mathbf{x})_k = - \sum_k w_k^{-2} b_k^2 = -2 \Phi_{\mathbf{w}}(\mathbf{x}). \quad (3)$$

The code exploits this identity directly: `dir_deriv = -2.0 * state.phi_x` at `nonlinear.jl:216`. The implication for Algorithm 4 is that the Armijo predicate $\hat{\Phi} \leq \Phi + c_1 \alpha D$ becomes $\hat{\Phi} \leq \Phi(1 - 2c_1 \alpha)$, which never requires an extra gradient evaluation. The identity holds *only* because the search direction is the exact Newton step; a Picard step in general violates it, which is one reason the Picard fallback in Algorithm 6 runs with its own (cruder) line-search settings.

4.2 Algorithm A: the safeguarded Newton outer loop

Algorithm 2: ExactNewtonPipeline: safeguarded Newton iteration with merit-based line search.

Parameters: $N_N, \tau_{\text{ftol}}, \tau_{\text{xtol}}, \tau_{\text{stag}}, c_1, \gamma, \alpha_{\min}, N_{\text{ls}}, F_{\text{div}}, N_{\text{inc}}$.

Input: Initial fields $\mathbf{x}^0 \in \mathcal{X}_h$, nonlinear operator op (encoding either an exact-Newton or a Picard Jacobian).

Output: $(\mathbf{x}^*, S \in \{\text{True}, \text{False}\}, I, \|\mathbf{b}^*\|_\infty, r)$, where r is the exit reason (Table 3).

```

b ← b( $\mathbf{x}^0$ ); if  $\|\mathbf{b}\|_\infty \leq \tau_{\text{ftol}}$  then
  | return ( $\mathbf{x}^0$ , True, 0,  $\|\mathbf{b}\|_\infty$ , InitialFtol);
end
 $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}^0$ ;  $b_{\text{best}} \leftarrow \|\mathbf{b}\|_\infty$ ;  $I_{\text{up}} \leftarrow 0$ ;  $r \leftarrow \text{IterationCap}$ ;
for  $i = 1, \dots, N_N$  do
  |  $\mathcal{J} \leftarrow \mathcal{J}(\mathbf{x})$ ; update weights  $\mathbf{w}$  via (2);  $\Phi \leftarrow \Phi_{\mathbf{w}}(\mathbf{x})$ ;
  |  $(S_{\text{LS}}^{\text{lin}}, \delta\mathbf{x}) \leftarrow \text{SolveLinearSystem}(\mathcal{J}, \mathbf{b})$ ; // Alg. 3
  | if  $S_{\text{LS}}^{\text{lin}} = \text{Failed}$  then
  | |  $\mathbf{x} \leftarrow \mathbf{x}_{\text{best}}$ ; return ( $\mathbf{x}$ , False,  $i$ ,  $b_{\text{best}}$ , LinearSolveNaN);
  | end
  | set  $D \leftarrow -2\Phi$ ;  $(S_{\text{LS}}, \alpha, L, \hat{b}, \hat{\Phi}) \leftarrow \text{ArmijoLinesearch}(\mathbf{x}, \mathbf{x}_{\text{old}}, \mathbf{b}, \delta\mathbf{x}, D, \mathbf{w}, \Phi)$ ; // Alg. 4
  | if  $\hat{b} < b_{\text{best}}$  then
  | |  $\mathbf{x}_{\text{best}} \leftarrow \mathbf{x}$ ;  $b_{\text{best}} \leftarrow \hat{b}$ ;
  | end
  |  $(r, S_{\text{conv}}, S_{\text{brk}}, I_{\text{up}}) \leftarrow \text{EvaluateSafeguards}(i, S_{\text{LS}}, L, \hat{b}, \hat{\Phi}, \Phi, I_{\text{up}})$ ; // Alg. 5
  | if  $S_{\text{brk}}$  then
  | | if  $\neg S_{\text{conv}}$  and  $r \notin \{\text{Ftol}, \text{NoiseFloor}, \text{Xtol}\}$  then
  | | |  $\mathbf{x} \leftarrow \mathbf{x}_{\text{best}}$ ;
  | | | end
  | | return ( $\mathbf{x}$ ,  $S_{\text{conv}}$ ,  $i$ ,  $\|\mathbf{b}^*\|_\infty$ ,  $r$ );
  | end
end
return ( $\mathbf{x}$ , False,  $N_N$ ,  $\|\mathbf{b}\|_\infty$ , IterationCap);

```

Reading the box. The outer loop is a textbook safeguarded Newton iteration: at each step compute the Jacobian, solve a linear system for the Newton direction, search along it for a step that decreases the merit, and ask the safeguard module whether to keep going. Three details are worth highlighting because they are easy to miss.

- *The safeguard’s role is not just to detect convergence.* It also detects *progress without convergence* (residual is dropping; keep going), *stuck progress* (step is shrinking faster than the residual; declare Xtol stagnation), and *merit explosion* (the line search succeeded but the new merit is far above the old; count an “increase” and abort if too many accumulate).
- *Best-state retention.* Whenever the residual strictly improves, $(\mathbf{x}_{\text{best}}, b_{\text{best}})$ is updated. On most non-converged exits the code restores the iterate to this best-seen point, so the caller never receives a state worse than what was already produced during the iteration. Three exits intentionally *do not* restore: Ftol and NoiseFloor are already at or below the target, and Xtol stagnation reflects a state the iterate could not move beyond—restoring would discard the most current information.
- *Exact-Newton vs Picard reuse.* The same loop is reused for the Picard fallback in Algorithm 6; the only difference is the Jacobian wired into op . The merit, line search, and safeguards apply unchanged. This is what makes the cascade in Algorithm 6 cheap to maintain.

4.3 Algorithm A.1: linear-system solve and NaN guard

Algorithm 3: SolveLinearSystem.

Parameters: (none).

Input: Jacobian $\mathcal{J}$, residual $\mathbf{b}$.

Output: ($S_{\text{Failed}} \in \{\text{True}, \text{False}\}$, $\delta\mathbf{x}$).

```

try { Solve  $\mathcal{J}\delta\mathbf{x} = \mathbf{b}$  via the configured linear solver } catch { return (True,  $\delta\mathbf{x}$ ) ;
  // linear-solver exception
}
if any element of  $\delta\mathbf{x}$  is NaN then
  | return (True,  $\delta\mathbf{x}$ ) ;                                // catastrophic algebraic failure
end
return (False,  $\delta\mathbf{x}$ );

```

The linear solve is the cheapest place to detect a near-singular Jacobian: either the factorisation aborts with an exception (the `try/catch` on `nonlinear.jl:54--61`) or it returns a vector contaminated by NaNs. Either way the outer loop must restore $\mathbf{x}_{\text{best}}$ and exit. Without this guard, a single NaN propagates through the residual evaluation and silently destroys the iterate.

4.4 Algorithm A.2: Armijo backtracking line search

Algorithm 4: ArmijoLinesearch.

Parameters: $\alpha_{\min}$, c_1 , γ , N_{ls} .

Input: $\mathbf{x}$, $\mathbf{x}_{\text{old}}$, $\mathbf{b}$, $\delta\mathbf{x}$, directional derivative $D = -2\Phi$, weights $\mathbf{w}$, current merit Φ .

Output: ($S_{\text{LS}} \in \{\text{True}, \text{False}\}$, α , L , $\|\hat{\mathbf{b}}\|_{\infty}$, $\hat{\Phi}$).

```

 $\alpha \leftarrow 1$ ;  $S_{\text{LS}} \leftarrow \text{False}$ ;
for  $j = 1, \dots, N_{\text{ls}}$  do
  |  $\mathbf{x} \leftarrow \mathbf{x}_{\text{old}} - \alpha \delta\mathbf{x}$ ;  $\hat{\mathbf{b}} \leftarrow \mathbf{b}(\mathbf{x})$ ;  $\hat{\Phi} \leftarrow \Phi_{\mathbf{w}}(\hat{\mathbf{b}})$ ;
  | if  $\hat{\Phi}$  is NaN then
  | |  $\alpha \leftarrow \gamma\alpha$ ; continue ;                                // shrink and retry without testing Armijo
  | end
  | if  $\hat{\Phi} \leq \Phi + c_1\alpha D$  then
  | |  $S_{\text{LS}} \leftarrow \text{True}$ ; break;
  | end
  | if  $\alpha \leq \alpha_{\min}$  then
  | | break ;                                                    // return with  $S_{\text{LS}} = \text{False}$ 
  | end
  |  $\alpha \leftarrow \gamma\alpha$ ;
end
 $L \leftarrow \alpha \|\delta\mathbf{x}\|_{\infty}$ ;
return ( $S_{\text{LS}}$ ,  $\alpha$ ,  $L$ ,  $\|\hat{\mathbf{b}}\|_{\infty}$ ,  $\hat{\Phi}$ );

```

What the parameters do.

- c_1 controls how aggressive the descent must be. Setting c_1 small (say 10^{-4}) accepts almost any step that lowers the merit at all; setting c_1 close to 1 demands a step that almost matches the full linearised decrease. Newton implementations typically choose $c_1 \sim 10^{-4}$ because the Newton direction itself is steep enough.
- $\gamma \in (0, 1)$ is the contraction factor. A typical choice is $\gamma = 1/2$; smaller γ tries fewer step sizes but explores more coarsely.
- $\alpha_{\min}$ is the smallest step size accepted before declaring the line search dead. Too small and the algorithm wastes iterations on machine-epsilon steps; too large and it gives up on legitimately stiff descent paths.

- N_{ls} caps the inner iteration count and is in practice the dominant cost knob: each iteration is a full residual evaluation.

NaN handling. When the residual evaluation at the trial point produces a NaN merit (typically because the Forchheimer factor $|\mathbf{u}|$ blew up under a too-aggressive step), the line search shrinks α without testing Armijo. This is critical near the quadratic-basin boundary: a Newton step pushing $\mathbf{u}$ into a region where the model momentarily becomes non-evaluable should not be recorded as a failed Armijo step; it should simply be retried at a smaller size.

4.5 Algorithm A.3: termination safeguards

Algorithm 5: EvaluateSafeguards.

Parameters: $\tau_{\text{ftol}}, \tau_{\text{stag}}, \tau_{\text{xtol}}, F_{\text{div}}, N_{\text{N}}, N_{\text{inc}}$.

Input: Outer index i , line-search success S_{LS} , step norm L , post-step residual $\|\hat{\mathbf{b}}\|_{\infty}$, post-step merit $\hat{\Phi}$, base merit Φ , increase counter I_{up} .

Output: $(r, S_{\text{conv}}, S_{\text{brk}}, I'_{\text{up}})$.

```

if  $\|\hat{\mathbf{b}}\|_{\infty} \leq \tau_{\text{ftol}}$  then
  | return (Ftol, True, True,  $I_{\text{up}}$ ) ;                                // converged on residual
end
if  $\neg S_{\text{LS}}$  then
  | if  $\|\hat{\mathbf{b}}\|_{\infty} \leq \tau_{\text{stag}}$  then
  | | return (NoiseFloor, True, True,  $I_{\text{up}}$ );
  | end
  | return (LineSearchFailed, False, True,  $I_{\text{up}}$ );
end
if  $\hat{\Phi} > F_{\text{div}} \Phi$  then
  |  $I_{\text{up}} \leftarrow I_{\text{up}} + 1$ ; if  $I_{\text{up}} \geq N_{\text{inc}}$  then
  | | return (MeritDivergence, False, True,  $I_{\text{up}}$ );
  | end
end
else
  |  $I_{\text{up}} \leftarrow 0$ ;
end
if  $L \leq \tau_{\text{xtol}}$  then
  | if  $\|\hat{\mathbf{b}}\|_{\infty} \leq \tau_{\text{stag}}$  then
  | | return (NoiseFloor, True, True,  $I_{\text{up}}$ );
  | end
  | return (Xtol, False, True,  $I_{\text{up}}$ );
end
if  $i = N_{\text{N}}$  then
  | if  $\|\hat{\mathbf{b}}\|_{\infty} \leq \tau_{\text{stag}}$  then
  | | return (NoiseFloor, True, True,  $I_{\text{up}}$ );
  | end
  | return (IterationCap, False, True,  $I_{\text{up}}$ );
end
return (InProgress, False, False,  $I_{\text{up}}$ );

```

In the implementation I_{up} is not returned explicitly but carried in the `inc_count` field of a mutable `SafeIterationState` struct that the same routine updates in place; the box treats it as a returned value for mathematical clarity.

Why six exit reasons? The safeguard logic distinguishes states that classical Newton implementations conflate, because in the context of the present cascade *the choice of reason changes what happens next*.

- Ftol and InitialFtol are unambiguous successes; the caller (Algorithm 6 or 1) proceeds to Stage II or exports results.

- **NoiseFloor** is a *conditional* success: the residual sits below τ_{stag} , which is itself far above floating-point noise but well below physical interest. The caller treats this as a success intentionally (Section 9 discusses the conditions under which this classification is and is not safe).
- **LinesearchFailed** and **MeritDivergence** both trigger the Picard fallback (Algorithm 6). They differ in cause: **LinesearchFailed** says the Newton direction itself is useful in direction but cannot find a length that descends; **MeritDivergence** says the Newton direction is actively pushing the iterate uphill.
- **Xtol** flags an iterate that has stopped moving even though the residual is non-trivial; the orchestrator treats this as failure unless the residual has slipped below τ_{stag} (in which case it is silently relabelled as **NoiseFloor**).
- **IterationCap** is the budget-exhausted failure. As with **Xtol**, a sufficiently small residual converts it to **NoiseFloor**.
- **LinearSolveNaN** aborts the loop unconditionally; the caller falls back to Picard, which may produce a non-singular linearisation.

The asymmetry between failures that promote to **NoiseFloor** and failures that do not is intentional: a stalled iterate *with a small residual* is a converged iterate that simply cannot be improved further by the inner solver, whereas **LinesearchFailed** and **MeritDivergence** never appear with a small residual—they are signals from the iteration map itself, not from how close the iterate is to the root.

The role of F_{div} and N_{inc} together. A single merit-expansion event is not a divergence: Newton near a saddle can take one or two steps that increase the merit before the quadratic basin captures the iterate. Requiring N_{inc} *consecutive* expansions, each by at least a factor F_{div} , separates this transient overshooting from a genuine repeated divergence. Choosing N_{inc} small (≤ 2) makes the guard trip almost immediately on hard problems and forces an early Picard hand-off; choosing it large (≥ 5) lets Newton recover from non-trivial overshoots but at the cost of diverging further before failing.

5 The Picard Fallback

When the safeguarded Newton loop terminates without converging, the orchestration does not give up: it transitions to a Picard iteration map and then re-engages Newton.

Algorithm 6: PicardFallback: the Stage I cascade.

Parameters: $N_N, N_P, \tau_{\text{ftol}}, \tau_{\text{stag}}$, all Newton/line-search parameters.

Input: Initial guess $\mathbf{x}^0 \in \mathcal{X}_h$ and a backup copy $\mathbf{x}_{\text{backup}}^0$.

Output: $(S, \mathbf{x}^*)$ where S is the overall success flag.

```

configure  $\pi_h \leftarrow 0$  ;                                     // ASGS-mode residual and Jacobian
 $(\mathbf{x}^*, S_N, \dots, r_N) \leftarrow \text{ExactNewtonPipeline}(\mathbf{x}^0, \text{ExactNewton Jacobian})$ ;
if  $S_N = \text{True}$  then
  | return  $(\text{True}, \mathbf{x}^*)$  ;                               // Newton sufficed; no fallback
end
restore  $\mathbf{x}^0 \leftarrow \mathbf{x}_{\text{backup}}^0$ ;
 $(\mathbf{x}^*, S_P, \dots, r_P) \leftarrow \text{ExactNewtonPipeline}(\mathbf{x}^0, \text{Picard Jacobian})$ ;
if any component of  $\mathbf{x}^*$  is non-finite then
  | return  $(\text{False}, \mathbf{x}_{\text{backup}}^0)$  ;                       // catastrophic state
end
if  $S_P = \text{True}$  then
  | return  $(\text{True}, \mathbf{x}^*)$ ;
end
/* Re-engage Newton from wherever Picard left the iterate.                                     */
 $(\mathbf{x}^*, S_{N2}, \dots, r_{N2}) \leftarrow \text{ExactNewtonPipeline}(\mathbf{x}^*, \text{ExactNewton Jacobian})$ ;
return  $(S_{N2} \vee S_P, \mathbf{x}^*)$ ;

```

Why Picard works where Newton fails. The Newton Jacobian for the porous Navier–Stokes system contains the linearised convective term $\mathbf{u}^k \cdot \nabla \delta \mathbf{u} + \delta \mathbf{u} \cdot \nabla \mathbf{u}^k$ and the linearised Forchheimer term $\partial \sigma / \partial \mathbf{u}$. Both of these become large in the difficult regimes, and the Jacobian can develop a near-null direction where a small change in $\mathbf{u}$ produces an outsize change in the residual. Newton near such a cusp produces a step that diverges. The Picard Jacobian drops the dependence of $\mathbf{u} \cdot \nabla \mathbf{u}$ and of σ on the increment $\delta \mathbf{u}$, retaining only the “frozen” coefficient form. The resulting iteration map is a contraction in a wider region than Newton’s quadratic basin, at the cost of linear (rather than quadratic) convergence rate. Picard does not converge to within τ_{ftol} in general—but it does push the iterate into a region where the subsequent Newton retry has a fighting chance.

Triggers. The original algorithm box in earlier drafts of this document restricted the Picard entry to two exit reasons. The current implementation is more permissive: *any* non-converged Newton exit triggers the fallback, including Xtol stagnation, IterationCap, the linear-solver exception path, and any uncaught Julia exception during the Newton call. This is appropriate because all of these states leave $\mathbf{x}^0$ outside the basin, and a Picard pass on top costs the same regardless of which symptom triggered the entry.

The catastrophic-state guard. Picard, while contractive in benign regimes, is not bounded in general. After the Picard call, the orchestration checks any(lisfinite, $\mathbf{x}^*$). A NaN or Inf in the iterate is a hard signal that even Picard could not keep the model evaluable, and the fallback restores the original backup and returns failure. Without this guard the subsequent Newton call would crash on its first residual evaluation.

Connection to Algorithm 1. Stage I of the orchestration is exactly one call to PicardFallback. The “Picard-smoothed” state produced by line 8 of Algorithm 6 is U_h^{ASGS} in Algorithm 1, and it is the seed for the OSGS staggered loop.

6 OSGS Staggered Relaxation

The orthogonal sub-grid scale formulation augments the discrete weak form with a stabilisation term that depends on the L^2 projection $\boldsymbol{\pi}_h$ of the strong residual. To avoid carrying the Fréchet derivative of Π_h in the global Jacobian, the algorithm treats the pair $(U_h, \boldsymbol{\pi}_h)$ by block-coordinate relaxation: freeze $\boldsymbol{\pi}_h$, solve the inner nonlinear primary system for U_h , then re-project to get the next $\boldsymbol{\pi}_h$.

6.1 Algorithm C

Algorithm 7: OSGSFractionalRelaxation: staggered fixed point on (U_h, π_h) .

Parameters: $N_{\text{OSGS}}, N_{\text{inner}}, m_w, \tau_w, \tau_{\text{ftol}}, \tau_{\text{OSGS}}, \tau_{\text{proj}}, \tau_{\text{stag}}, \text{Mode}_{\text{drift}}, \text{Mode}_{\text{stop}}$.

Input: Stage-I converged state U_h^{ASGS} ; residual operators $(\mathcal{R}_u, \mathcal{R}_p)$; pre-factored mass solvers (M_u^{-1}, M_p^{-1}) ; optional Anderson accelerators.

Output: $(S, \pi_{h,u}, \pi_{h,p}, \dots)$.

```

 $U_h^0 \leftarrow U_h^{\text{ASGS}};$ 
 $\pi_{h,u}^0 \leftarrow \Pi_{V_{\text{free}}}(\mathcal{R}_u(U_h^0)); \quad \pi_{h,p}^0 \leftarrow \Pi_{Q_{\text{free}}}(\mathcal{R}_p(U_h^0));$ 
for  $m = 1, \dots, N_{\text{OSGS}}$  do
    set effective inner tolerance  $\tau_{\text{ftol},m}^{\text{inner}} \leftarrow (m \leq m_w) ? \max(\tau_{\text{ftol}}, \tau_w) : \tau_{\text{ftol}};$ 
    fix  $\pi_h \leftarrow \pi_h^{m-1}; \quad (U_h^m, S_m^{\text{inner}}, \dots, r_m^{\text{inner}}) \leftarrow \text{ExactNewtonPipeline}(U_h^{m-1})$  with cap  $N_{\text{inner}}$ 
    and tolerance  $\tau_{\text{ftol},m}^{\text{inner}};$ 
    if  $r_m^{\text{inner}} \in \{\text{LinesearchFailed}, \text{MeritDivergence}, \text{LinearSolveNaN}\}$  then
        | return  $(\text{False}, \pi_{h,u}^{m-1}, \pi_{h,p}^{m-1}, \dots);$  // abort: structural OSGS divergence
    end
    /* State drift, in the configured metric. */
    if  $\text{Mode}_{\text{drift}} = \text{L2.mass}$  then
        |  $d_{\text{state}}^m \leftarrow \max(\|u_h^m - u_h^{m-1}\|_{L^2(\Omega)}, \|p_h^m - p_h^{m-1}\|_{L^2(\Omega)});$ 
    end
    else
        |  $d_{\text{state}}^m \leftarrow \|X_h^m - X_h^{m-1}\|_{\ell^\infty}$  over the global DOF vector;
    end
    re-project:  $\pi_{h,u}^m \leftarrow \Pi_{V_{\text{free}}}(\mathcal{R}_u(U_h^m)); \quad \pi_{h,p}^m \leftarrow \Pi_{Q_{\text{free}}}(\mathcal{R}_p(U_h^m));$ 
    /* Optional block-wise Anderson acceleration on each projection. */
    if accelerator present then
        |  $\pi_{h,u}^m \leftarrow \text{Anderson}_u(\pi_{h,u}^{m-1}, \pi_{h,u}^m); \quad \pi_{h,p}^m \leftarrow \text{Anderson}_p(\pi_{h,p}^{m-1}, \pi_{h,p}^m);$ 
    end
     $d_\pi^m \leftarrow \max(\sqrt{(\Delta\pi_{h,u}^m)^\top M_u (\Delta\pi_{h,u}^m)}, \sqrt{(\Delta\pi_{h,p}^m)^\top M_p (\Delta\pi_{h,p}^m)});$ 
    /* Convergence predicate, noise-floor protected. */
     $\tau_{\text{OSGS}}^{\text{eff}} \leftarrow \max(\tau_{\text{OSGS}}, \tau_{\text{stag}}); \quad \tau_{\text{proj}}^{\text{eff}} \leftarrow \max(\tau_{\text{proj}}, \tau_{\text{stag}});$ 
     $S_{\text{state}} \leftarrow (d_{\text{state}}^m \leq \tau_{\text{OSGS}}^{\text{eff}}); \quad S_{\text{proj}} \leftarrow (d_\pi^m \leq \tau_{\text{proj}}^{\text{eff}});$ 
     $S_{\text{conv}} \leftarrow \begin{cases} S_{\text{state}} & \text{Mode}_{\text{stop}} = \text{state.drift}, \\ S_{\text{proj}} & \text{Mode}_{\text{stop}} = \text{projection.drift}, \\ S_{\text{state}} \wedge S_{\text{proj}} & \text{Mode}_{\text{stop}} = \text{both}. \end{cases}$ 
    if  $m \geq 2$  and  $\|X_h^m - X_h^{m-1}\|_{\ell^\infty} \leq \tau_{\text{stag}}$  then
        |  $S_{\text{conv}} \leftarrow \text{True};$  // absolute noise-floor short circuit
    end
    if  $S_{\text{conv}}$  then
        | [if MMS enabled, run VerifyMMSPlateau (Alg. 8)];
        | return  $(\text{True}, \pi_{h,u}^m, \pi_{h,p}^m, \dots);$ 
    end
end
return  $(\text{False}, \pi_{h,u}^{N_{\text{OSGS}}}, \pi_{h,p}^{N_{\text{OSGS}}}, \dots);$ 

```

6.2 The bootstrap and why the ASGS state matters

The first sub-grid projection π_h^0 is the L^2 projection of the strong residual evaluated at U_h^{ASGS} , not at the zero initial guess. There are two reasons for this.

First, when the ASGS system has converged, $\mathcal{R}_u(U_h^{\text{ASGS}})$ and $\mathcal{R}_p(U_h^{\text{ASGS}})$ are small in L^2 , so π_h^0 is already close to its fixed-point value. The staggered loop is then a perturbation around an existing solution rather than a global search.

Second, an ASGS-converged primary field gives the inner Newton solve at $m = 1$ an initial residual that is small in absolute terms, so the Newton iteration usually converges within a handful of inner

iterations. Starting OSGS from $\mathbf{x}^0 = 0$, by contrast, would force the solver to chase both the primary nonlinearity and the projection coupling simultaneously, which empirically diverges in stiff regimes.

6.3 Why projection happens on the unconstrained spaces

The projection target $V_{\text{free}}, Q_{\text{free}}$ has no Dirichlet constraints. If the projection were performed onto the constrained trial spaces U, P , the boundary residual—which is non-zero in general—would be assigned to the projection’s boundary DOFs, producing an artefact of order $O(1)$ along $\partial\Omega$. That artefact then enters the stabilisation term, where it pollutes the convergence rate of the FE method to $O(1)$ rather than the expected $O(h^{k+1})$, destroying any MMS rate verification. See `theory/paper-code-divergences.md`.

6.4 Mass-matrix pre-factoring

Each projection $\Pi_{V_{\text{free}}}, \Pi_{Q_{\text{free}}}$ is the solution of an SPD linear system with the mass matrices M_u, M_p . Because the spaces $V_{\text{free}}, Q_{\text{free}}$ do not change between outer iterations, the matrices and their LU factorisations are assembled *once* (Algorithm 1, line 16). At every outer iteration the projection cost is then one forward and one back substitution per block; the cubic factorisation cost is amortised over the full N_{OSGS} budget.

6.5 Warmup phase

For the first m_w outer iterations the inner Newton tolerance is relaxed to $\max(\tau_{\text{ftol}}, \tau_w)$. The motivation is purely economic: while π_h^0 is still being computed off a not-yet-stationary residual, driving the inner solve to a tight τ_{ftol} “polishes” the iterate against a projection that will be replaced immediately. Relaxing the inner tolerance during this phase trades a tiny amount of accuracy in the early iterates for a noticeable cut in inner-Newton budget. Once $m > m_w$ the inner tolerance reverts to the global τ_{ftol} and the staggered loop tightens.

A practical default has $m_w \in \{1, 2, 3\}$ and $\tau_w = 10 \cdot \tau_{\text{ftol}}$ or so. Increasing m_w further wastes fewer Newton iterations early but risks the staggered fixed point “locking” on a coarse projection.

6.6 State drift: ℓ^∞ vs continuous L^2

The state drift between consecutive outer iterations

$$d_{\text{state}}^m = \begin{cases} \max(\|\mathbf{u}_h^m - \mathbf{u}_h^{m-1}\|_{L^2(\Omega)}, \|p_h^m - p_h^{m-1}\|_{L^2(\Omega)}) & \text{Mode}_{\text{drift}} = \text{L2_mass}, \\ \|\mathbf{X}_h^m - \mathbf{X}_h^{m-1}\|_{\ell^\infty} & \text{Mode}_{\text{drift}} = \text{Linf}, \end{cases} \quad (4)$$

admits two metrics with subtly different meanings.

The Linf mode. This is the cheapest possible drift: the raw vector-space ℓ^∞ norm of the concatenated free-DOF array. It costs one BLAS sweep and no quadrature. The pathology, however, is that the same array stores velocity and pressure DOFs interleaved by the multi-field layout, so a velocity DOF expressed in (dimensional) units of [m/s] sits beside a pressure DOF in [Pa]. The ℓ^∞ norm is then dominated by whichever field happens to have the larger numerical magnitude—a scale-mixing artefact, not a physical drift measurement. In regimes where pressure and velocity are well balanced this mode works adequately and is fast. In regimes where one field dominates, the drift signal can become essentially blind to the other.

The L2_mass mode. Computes two independent continuous $L^2(\Omega)$ norms via quadrature, one for each block, and takes their maximum. This is the recommended default (and the invariant codified in `CLAUDE.md`): velocity and pressure are measured in their own natural functional norms, with no possibility of cross-pollution. The cost is two mass-weighted quadratures per outer iteration, dominated in practice by the inner Newton solve itself.

6.7 Projection drift, in the mass-weighted inner product

The projection drift

$$d_\pi^m = \max\left(\sqrt{(\Delta\pi_{h,u}^m)^\top M_u (\Delta\pi_{h,u}^m)}, \sqrt{(\Delta\pi_{h,p}^m)^\top M_p (\Delta\pi_{h,p}^m)}\right) \quad (5)$$

is the natural distance in the inner product that defines the L^2 projection itself. Because the projection is the minimiser of a quadratic in M , the appropriate measure of progress toward the fixed point is the M -weighted norm; the Euclidean DOF norm would conflate it with inverse-mass scaling and report misleading drift values on graded meshes. The factorisations M_u^{-1}, M_p^{-1} are not needed for this norm; only the matrix-vector products $M_u \Delta \pi, M_p \Delta \pi$ are required, and those reuse the same assembled sparse matrices used in the projection step.

6.8 Choosing the stopping mode

The user selects `Modestop` between three options:

- **state_drift**: terminate when $d_{\text{state}}^m \leq \tau_{\text{OSGS}}^{\text{eff}}$. Use when the primary fields are the only quantities of interest and the projection is a means to an end.
- **projection_drift**: terminate when $d_{\pi}^m \leq \tau_{\text{proj}}^{\text{eff}}$. Use when sub-grid scale accuracy is the goal and one expects the primary field to be approximately stationary already.
- **both** (default): require both predicates. The most defensive option and the one that aligns with the mathematical definition of the fixed point.

In all three cases an additional absolute short-circuit fires when the raw ℓ^∞ DOF drift drops below τ_{stag} for $m \geq 2$; this protects against limit-cycle oscillations in which the iterate ping-pongs between two nearby states without either drift ever cleanly dropping below τ_{OSGS} .

6.9 Block-wise Anderson acceleration

For high-Re, high-Da flows the staggered fixed point on π_h can stall in slow limit-cycle oscillations. Anderson acceleration extrapolates a new π_h^m from the history of the last m fixed-point residuals by solving a small mass-weighted least-squares problem

$$\min_{\gamma} \|(\Delta F)\gamma - f_k\|_M^2, \quad (6)$$

where ΔF is the matrix of residual increments and the inner product $\|\cdot\|_M^2 = (\cdot)^\top M(\cdot)$ uses the appropriate mass matrix. Two architectural choices make this robust in the present setting:

Block isolation. The accelerator operates independently on $\pi_{h,u}$ and $\pi_{h,p}$, each with its own mass matrix (M_u and M_p respectively). Combining the blocks into a single extrapolation problem would force a cross-physical mixing that the OSGS formulation goes out of its way to avoid in the first place.

Mass-weighted normal equations. The least-squares system uses the same L^2 inner product that defines the underlying projection, not the Euclidean dot product of DOF vectors. This is the natural way to make the extrapolation respect mesh grading.

Safety fallback. The implementation contains a sanity check: if the extrapolated state x_{next} lies further than 10 times the unextrapolated residual from the raw fixed-point candidate g_k , the extrapolation is discarded and the iteration falls back to a relaxed Picard step $x_k + \beta f_k$. The factor 10 is a conservative bound chosen so that the safety net does not interfere with legitimate Anderson acceleration on well-behaved problems while still catching catastrophic over-extrapolation in degenerate cases.

6.10 The reaction-projection trim

When the configured `experimental_reaction_mode` is "standard" and the reaction law is `ConstantSigmaLaw`, the projected residual omits the σu term entirely (`src/stabilization/projection.jl:26–40`). Mathematically this is the observation that for constant σ the L^2 projection of σu onto the FE space is itself, so the contribution to $\Pi_h(\mathcal{R}_u)$ exactly cancels in the OSGS residual. Skipping it saves an assembly per outer iteration without altering the converged state. For non-constant reaction laws (e.g. Forchheimer–Ergun) the trim is invalid and the projection-policy sanitiser rejects this option.

6.11 Failure of the inner Newton

The inner Newton call inside Algorithm 7 runs on the same ExactNewtonPipeline used in Algorithm 2, but with the iteration cap reduced to N_{inner} (typically a small number, e.g. 3–5) and the tolerance possibly relaxed during warmup. When the inner exit reason is one of `LinesearchFailed`, `MeritDivergence`, or `LinearSolveNaN`, the staggered outer loop aborts immediately: these reasons reflect structural failure of the inner iteration map, and continuing the outer relaxation would only re-feed the broken state into another projection. Other exit reasons (notably `IterationCap` from exhausting N_{inner} without converging to the effective inner tolerance) are treated as progress: the next outer iteration simply re-engages with the updated projection.

7 MMS Plateau Verification

7.1 Numerical convergence versus discretisation convergence

The inner Newton stops when $\|\mathbf{b}\|_{\infty} \leq \tau_{\text{ftol}}$. That is convergence in the *algebraic* sense: the iterate is close to the root of the discrete system. It says nothing about whether the discrete system itself is close to the true PDE solution. The Method of Manufactured Solutions (MMS) provides exactly that second test: it injects an exact analytical $(\mathbf{u}_{\text{ex}}, p_{\text{ex}})$, recomputes the appropriate source terms, and measures $\|U_h - U_{\text{ex}}\|$ in physical norms. The expected discretisation behaviour is $O(h^{k+1})$ in the L^2 norms (and one order weaker in the H^1 -semi norm).

For a single mesh, plateau verification asks a different question: once the iterate is close enough to the root that further iterations do not move the FE error, the FE error has *plateaued* at its discretisation floor. Detecting this plateau certifies that the algebraic convergence criterion was tight enough to expose the discretisation error.

7.2 Algorithm D

Algorithm 8: VerifyMMSPlateau: relative-change plateau test.

Parameters: $N_{\text{plat}}, \tau_{\text{err}}, \varepsilon_{u,L^2}, \varepsilon_{u,H^1}, \varepsilon_{p,L^2}$.

Input: Current iterate U_h^m , exact reference U_{ex} , history $\mathcal{H}$ of past error tuples.

Output: (S_{plat}) .

```

compute  $E_{u,L^2}^m = \|\mathbf{u}_h^m - \mathbf{u}_{\text{ex}}\|_{L^2}$ ,  $E_{u,H^1}^m = \|\mathbf{u}_h^m - \mathbf{u}_{\text{ex}}\|_{H^1}$ ,  $E_{p,L^2}^m = \|p_h^m - p_{\text{ex}}\|_{L^2}$ ;
append to  $\mathcal{H}$ ; let  $E_{\bullet}^{m-1}$  be the previous entry;
 $r_{u,L^2} \leftarrow |E_{u,L^2}^m - E_{u,L^2}^{m-1}| / \max(E_{u,L^2}^m, E_{u,L^2}^{m-1}, \varepsilon_{u,L^2})$ ;
 $r_{u,H^1} \leftarrow |E_{u,H^1}^m - E_{u,H^1}^{m-1}| / \max(E_{u,H^1}^m, E_{u,H^1}^{m-1}, \varepsilon_{u,H^1})$ ;
 $r_{p,L^2} \leftarrow |E_{p,L^2}^m - E_{p,L^2}^{m-1}| / \max(E_{p,L^2}^m, E_{p,L^2}^{m-1}, \varepsilon_{p,L^2})$ ;
 $r_{\text{max}} \leftarrow \max(r_{u,L^2}, r_{u,H^1}, r_{p,L^2})$ ;
if  $r_{\text{max}} \leq \tau_{\text{err}}$  then
    |  $I_{\text{pass}} \leftarrow I_{\text{pass}} + 1$ ;
end
else
    |  $I_{\text{pass}} \leftarrow 0$ ;
end
if  $I_{\text{pass}} \geq N_{\text{plat}}$  then
    | return True ;                                     // plateau satisfied
end
return False ;                                         // continue extending

```

7.3 Three norms, three floors

The verifier tracks *three* error quantities, not two: L^2 of velocity, L^2 of pressure, and the H^1 -semi-norm $\|\mathbf{u}_h - \mathbf{u}_{\text{ex}}\|_{H^1} = \sqrt{\int_{\Omega} \nabla(\mathbf{u}_h - \mathbf{u}_{\text{ex}}) : \nabla(\mathbf{u}_h - \mathbf{u}_{\text{ex}}) \, d\Omega}$. The third norm matters because pressure L^2 and velocity L^2 can plateau while the velocity gradient—which is what the convective and viscous terms

actually feel—still drifts. Without the H^1 -semi norm a silent gradient drift would be invisible to the plateau test.

Each ratio is taken with respect to its own per-norm floor $\varepsilon_\bullet$. Without these floors, when an error first drops to 10^{-15} the ratio between consecutive iterations becomes numerically meaningless. The floors are configured high enough above floating-point noise to remain stable on fine meshes, but low enough that they do not artificially declare convergence for genuinely coarse runs.

7.4 Consecutive passes and budget exhaustion

The plateau is declared only after N_{plat} *consecutive* passes, and the counter resets to zero on the first failure. This catches intermittent drift—e.g. a sequence $(\tau_{\text{max}}^k)_k = (10^{-4}, 10^{-3}, 10^{-4}, \dots)$ that looks safe sample-by-sample but actually has not stabilised. A typical choice is $N_{\text{plat}} \in \{2, 3\}$.

7.5 Two entry points and how they differ

The verifier is invoked from two distinct contexts.

ASGS path. After Stage I converges and the method is ASGS, the orchestration enters a dedicated extension loop of length K_{extra} (`solve_system`, lines 258–337). Each iteration calls `ExactNewtonPipeline` with $N_N = 1$ followed by `VerifyMMSPlateau`. The loop exits when the plateau passes or the budget is exhausted; either way the iterate is returned. The “one inner Newton step then verify” cadence is deliberate: it gives the plateau test a chance to observe how a single extra Newton step changes the FE error.

OSGS path. Within `OSGSFractionalRelaxation` (Algorithm 7), the verifier runs only after the staggered predicate S_{conv} has fired, not at every outer iteration. The first successful staggered iteration triggers a fresh MMS error history; subsequent successful iterations append to it and test for the plateau. The budget inflation K_{extra} allows the staggered loop to keep producing additional converged iterates after first hitting the fixed point. This entry point is gated rather than per-iteration because the FE error is defined relative to the discrete system, and that system itself is moving between staggered iterations until π_h stabilises.

8 The MMS Sweep Harness

The discussion above applies to a single `run_simulation` call. The MMS sweep harness under `test/extended/ManufacturedSolutions/` wraps many `run_simulation` runs in a triple loop over Reynolds number Re , Damköhler number Da , and mesh resolution h , and reports convergence-rate tables across the resulting sweep. Because the harness sees many cases of widely varying difficulty, it injects three per-case adaptations that the main `run_simulation` flow deliberately does not attempt. We describe them here in one place so that the reader does not mistake them for parts of the orchestration.

8.1 Adaptive τ_{ftol}

Two error scales matter in an FE method. The *algebraic error* is the distance between the iterate and the discrete root, governed by τ_{ftol} . The *discretisation error* is the distance between the discrete root and the true PDE solution, which behaves as $O(h^{k_v+1})$ for the chosen velocity element order. Setting τ_{ftol} much tighter than $O(h^{k_v+1})$ wastes Newton iterations driving the algebraic residual far below the discretisation floor—adjustments that cannot improve the discretisation error. Setting τ_{ftol} much looser than $O(h^{k_v+1})$ lets the algebraic error dominate and pollutes the observed convergence rate.

The harness therefore re-derives τ_{ftol} per case:

$$\tau_{\text{ftol}}^{\text{dyn}} = \max\left(\tau_{\text{ftol}}^{\text{static}}, \min(C_{\text{ceil}}, C_{\text{sf}} \cdot h^{k_v+1})\right). \quad (7)$$

The right-most term ties the tolerance to the discretisation scale via a spatial safety factor C_{sf} (a typical choice is 10^{-2} , i.e. the algebraic residual must drop one to two orders of magnitude below the predicted discretisation error). The ceiling C_{ceil} prevents trivially loose tolerances on absurdly coarse meshes. The outer max prevents the formula from descending below the user-supplied $\tau_{\text{ftol}}^{\text{static}}$, which serves as a sanity

bound. The adaptive $\tau_{\text{ftol}}^{\text{dyn}}$ is then plugged into every `SafeNewtonSolver` the harness constructs for that case (`run.test.jl`:440–446, 470–471).

The main `run_simulation` driver does not use this scaling because in production work the user typically picks one mesh and one tolerance deliberately, and an adaptive rule would silently override that choice.

8.2 Adaptive noise floor

Linear-system conditioning grows as $\mathcal{O}(h^{-2})$ for elliptic operators and is further amplified by convective and reactive terms. A static τ_{stag} that is appropriate on a coarse mesh will be far above the genuine round-off floor on a fine mesh, causing the `NoiseFloor` classification of Algorithm 5 to fire prematurely. The harness consequently scales

$$\tau_{\text{stag}}^{\text{dyn}} = \min\left(\tau_{\text{stag}}, \max(n^{\min}, n^{\text{base}} \cdot N^2 \cdot \max(1, \text{Re}))\right), \quad (8)$$

clipped above by the user-supplied τ_{stag} and below by an absolute floor $n^{\min}$. The factor N^2 tracks the elliptic conditioning growth (with N the partition count per direction, so $h \propto 1/N$); the $\max(1, \text{Re})$ factor accounts for convective amplification. A final $\tau_{\text{stag}}^{\text{dyn}} \leftarrow \max(\tau_{\text{stag}}^{\text{dyn}}, n^{\text{sf}} \tau_{\text{ftol}}^{\text{dyn}})$ clamp preserves the strict ordering $\tau_{\text{stag}}^{\text{dyn}} > \tau_{\text{ftol}}^{\text{dyn}}$, without which Algorithm 5 would collapse the two predicates.

8.3 Per-case Picard widening

In a $(\text{Re}, \text{Da}, h)$ sweep, the easy cases require few Picard iterations and the hard ones need many. Setting a uniform N_{P} across the sweep forces an unhappy choice: large enough for the hard cases (wastefully slow on the easy ones) or small enough for the easy ones (insufficient on the hard ones). The harness therefore *widens* the Picard budget when a stiffness threshold is exceeded (`run.test.jl`:459–468):

$$\begin{aligned} N_{\text{P}}^{\text{local}} &\leftarrow N_{\text{P}}, \\ \text{if } \text{Re} \geq \text{Re}^{\text{th}} : & \quad N_{\text{P}}^{\text{local}} \leftarrow \max(N_{\text{P}}^{\text{local}}, N_{\text{P}}^{\text{Re}}), \\ \text{if } \text{Da} \geq \text{Da}^{\text{th}} : & \quad N_{\text{P}}^{\text{local}} \leftarrow \max(N_{\text{P}}^{\text{local}}, N_{\text{P}}^{\text{Da}}). \end{aligned} \quad (9)$$

$N_{\text{P}}^{\text{local}}$ is then plugged into the local `SafeNewtonSolver` for that case. The defaults $\text{Re}^{\text{th}} = \text{Da}^{\text{th}} = 10^4$ are crude break-points between regimes where the convective or reactive coupling becomes the dominant difficulty.

8.4 Closing remark

None of the adaptations of Sections 8.1–8.3 fire inside a single `run_simulation` call. The configuration schema exposes them so that the harness can read them from a shared JSON file, but the production driver simply uses the static τ_{ftol} , τ_{stag} , and N_{P} written in the user’s configuration.

9 Convergence-Analysis Notes

9.1 The false-equivalency failsafe

The single most important invariant of Algorithm 5 is the asymmetry between two exit reasons that, at first glance, look interchangeable:

- `IterationCap` with $\|\mathbf{b}\|_{\infty} \gg \tau_{\text{stag}}$: the iteration ran out of budget while still far from any noise floor.
- `NoiseFloor` with $\|\mathbf{b}\|_{\infty} \leq \tau_{\text{stag}}$: the iteration may have stopped for any reason, but the residual is below the noise floor we set for “effectively converged”.

The orchestration treats only the second as success. Coercing the first into a converged state—“it ran for many iterations and stopped; close enough”—is forbidden, because doing so would feed an unresolved iterate into Stage II of the cascade. In OSGS in particular, an unresolved U_h produces a stale π_h whose subsequent fixed-point iterations cannot recover.

To make this concrete: imagine a coarse-mesh MMS run in which Newton stops at `IterationCap` with $\|\mathbf{b}\|_{\infty} = 10^{-3}$, above $\tau_{\text{stag}} = 10^{-5}$ but well above $\tau_{\text{ftol}} = 10^{-10}$. The discretisation error on that mesh

might be 10^{-4} . If `IterationCap` were silently relabelled as converged, the verifier in Algorithm 8 would compute relative-change ratios that reflect a non-stationary iterate, the plateau test would either oscillate or fail spuriously, and the reported $O(h^{k+1})$ rate would be corrupted. The asymmetry in Algorithm 5 prevents exactly this.

9.2 Why τ_{stag} must scale with conditioning

The same invariant explains why the harness adapts τ_{stag} to the mesh and to Re (Section 8.2). On a fine mesh the legitimate machine-noise floor lies well below the coarse-mesh value, and a fixed τ_{stag} would silently rule out states that, while not at τ_{ftol} , are genuinely as close to it as the floating-point arithmetic allows. The scaling $\tau_{\text{stag}} \propto N^2 \cdot \max(1, \text{Re})$ tracks the condition number well enough to keep the `NoiseFloor` classification meaningful across the sweep.

9.3 Why the OSGS stopping mode matters

Algorithm 7 accepts three convergence predicates. The default `both` is the strictest and corresponds to the mathematical definition of the staggered fixed point: (U_h, π_h) is the converged pair when neither moves between outer iterations. The relaxed modes `state_drift` and `projection_drift` sacrifice one of the two conditions in exchange for fewer iterations. They are appropriate only when the user knows that the slacker condition is already satisfied—e.g. because the projection naturally lies in a regime where it has stabilised quickly, but the primary fields are still adjusting. Using a single-criterion mode without that knowledge introduces exactly the false-convergence trap that the noise-floor invariant forbids: the staggered loop declares victory while one of the two quantities is still drifting.

A Code–Notation Correspondence

Table 3 maps every exit-reason label used by the algorithm boxes to the string literal returned by the implementation.

The `SafeNewtonSolver` struct in `src/solvers/nonlinear.jl` declares all Newton-side numerical parameters explicitly via its constructor signature; the `StabilizationConfig` struct in `src/config.jl` declares the OSGS-side parameters; the `AcceleratorConfig` sub-struct declares the Anderson parameters. Every parameter listed in Table 1 corresponds to exactly one field across these three structures, validated by the `validate!` routine of `src/config.jl`; the routine rejects unknown JSON keys and enforces strict ranges (e.g. $0 < c_1 < 1$, $F_{\text{div}} \geq 1$, $\alpha_{\text{min}} \in (0, 1]$).

Table 1: Numerical parameters of the nonlinear solver pipeline. The “Effect” column summarises the dominant influence of each parameter on robustness or cost. Each parameter participates in every `run_simulation` call except those flagged *harness-only*; see Section 8.

Symbol	Field in code	Role / effect
<i>Inner Newton iteration controls</i>		
N_N	<code>sol.newton.iterations</code>	Cap on inner Newton iterations.
N_P	<code>sol.picard.iterations</code>	Cap on Picard fallback iterations.
N_{inc}	<code>sol.max_increases</code>	Consecutive merit-function expansions tolerated before declaring divergence. Smaller is stricter; larger is more permissive at the cost of slower failure detection.
τ_{ftol}	<code>sol.ftol</code>	Terminal residual tolerance in ℓ^∞ ; the dominant convergence criterion.
τ_{xtol}	<code>sol.xtol</code>	Lower bound on the accepted step norm $\ \alpha\delta\mathbf{x}\ $ before declaring coordinate stagnation.
τ_{stag}	<code>sol.stagnation_noise_floor</code>	Noise-floor residual; a non-converged exit below this may still be classified as success (Sec. 9).
α_{min}	<code>sol.linesearch.alpha_min</code>	Smallest line-search step before giving up. Smaller means more aggressive descent, but more failed Newton iterations per outer call.
c_1	<code>sol.armijo.c1</code>	Armijo descent constant. Smaller means easier step acceptance and a weaker guarantee.
γ	<code>sol.linesearch.contraction_factor</code>	Backtracking factor: $\alpha \leftarrow \gamma\alpha$. Smaller tries fewer step sizes but explores more coarsely.
N_{ls}	<code>sol.max_linesearch.iterations</code>	Bound on line-search inner iterations.
F_{div}	<code>sol.divergence_merit_factor</code>	Threshold by which the merit must expand to count as one “increase”. Smaller is a stricter divergence detector.
<code>freeze_cusp</code>	<code>sol.freeze_jacobian_cusp</code>	Boolean: skip Jacobian recomputation at Forchheimer cusps to avoid singular updates.
<i>OSGS staggered outer loop</i>		
N_{OSGS}	<code>stab.osgs.iterations</code>	Cap on outer (staggered) iterations.
N_{inner}	<code>stab.osgs.inner_newton_iters</code>	Cap on inner Newton iterations per outer step. Smaller makes each sweep cheap and forces more outer iterations; larger means a deeper inner solve and fewer outer iterations.
τ_{OSGS}	<code>stab.osgs.tolerance</code>	Tolerance on the state drift d_{state}^m .
τ_{proj}	<code>stab.osgs.projection_tolerance</code>	Tolerance on the mass-weighted projection drift d_{π}^m .
m_w	<code>stab.osgs.warmup.iterations</code>	Outer iterations during which the inner Newton uses a relaxed tolerance, to avoid polishing iterates against a stale projection.
τ_w	<code>stab.osgs.warmup.tolerance</code>	Relaxed inner tolerance during warmup; the effective inner tolerance is $\max(\tau_{\text{ftol}}, \tau_w)$.
$\text{Mode}_{\text{drift}}$	<code>stab.osgs.state_drift_scale</code>	Drift metric: Linf (raw ℓ^∞ on DOFs) or L2_mass (per-block continuous $L^2(\Omega)$). See Sec. 6.6.
$\text{Mode}_{\text{stop}}$	<code>stab.osgs.stopping_mode</code>	Convergence predicate: state_drift , projection_drift , or both (conjunction; the default).
m, β	<code>sol.accelerator.{m,relaxation_factor}</code>	Anderson history depth and damping. The accelerator is enabled by setting <code>sol.accelerator.type</code> to “Anderson”.
<i>MMS plateau verification</i>		
N_{plat}	<code>mms.require_consecutive_passes</code>	Consecutive passes of $r_{\text{max}} \leq \tau_{\text{err}}$ required before declaring a plateau.
τ_{err}	<code>mms.tau_err</code>	Relative-change tolerance against which r_{max} is tested.
$\epsilon_{u,L^2}, \epsilon_{u,H^1}, \epsilon_{p,L^2}$	<code>mms.eps_u_l2, u_h1, p_l2</code>	Per-norm floors guarding the denominator in r_{max} .
K_{extra}	<code>mms.max_extra_cycles</code>	Cap on extension cycles (ASGS path) or staggered iterations added to the OSGS budget (OSGS path) for plateau verification.
<i>Harness-only adaptations</i> (Section 8)		

Table 2: Algorithm $\leftrightarrow$ code signature mapping.

Algorithm	Code entry point
SimulationOrchestration (Alg. 1)	<code>run_simulation(path) → solve_system(setup, form, solvers, cfg, x0)</code> (<code>src/run_simulation.jl</code> , <code>src/solvers/porous_solver.jl</code>).
ExactNewtonPipeline (Alg. 2)	<code>Gridap.Algebra.solve!(x, FESolver(SafeNewtonSolver(...)), op)</code> dispatching into <code>_safe_solve_inner!</code> (<code>src/solvers/nonlinear.jl</code>). The same routine is reused with an <code>ExactNewtonMode()</code> Jacobian or a Picard Jacobian; the safeguard logic does not change.
SolveLinearSystem (Alg. 3)	<code>eval_linear_system_resolution!(dx, A, b, solver, cache).</code>
ArmijoLinesearch (Alg. 4)	<code>eval_armijo_linesearch_pass!(x, x_old, b, dx, op, dir_deriv, solver, w, state).</code>
EvaluateSafeguards (Alg. 5)	<code>eval_safeguard_termination_bounds!(solver, state).</code>
PicardFallback (Alg. 6)	The Newton→Picard→Newton cascade in <code>solve_system</code> , lines 169–246 of <code>src/solvers/porous_solver.jl</code> .
OSGSFractionalRelaxation (Alg. 7)	The <code>for osgs_iter in 1:eff_osgs_iters</code> block in <code>solve_system</code> , lines 340–615.
VerifyMMSPlateau (Alg. 8)	The extended-cycle loop in <code>solve_system</code> (ASGS entry, lines 258–337) and the in-loop plateau gate (OSGS entry, lines 546–587).

Table 3: Exit reasons in Algorithm 5 and their code-side strings.

Box label	Code string	Meaning
Ftol	"ftol_reached"	$\ \mathbf{b}\ _\infty \leq \tau_{\text{ftol}}$ after a successful line search.
InitialFtol	"initial_ftol"	Initial guess already satisfies τ_{ftol} ; no Newton step taken.
NoiseFloor	"stagnation_noise_floor_reached"	$\ \mathbf{b}\ _\infty \leq \tau_{\text{stag}}$ on a non-converged exit; treated as success.
LinesearchFailed	"linesearch_failed"	Line-search step shrank to $\alpha \leq \alpha_{\min}$ without descent.
MeritDivergence	"merit_divergence_escaped"	Merit function expanded by $\geq F_{\text{div}}$ for N_{inc} consecutive steps.
Xtol	"xtol_stagnation"	Step norm $L \leq \tau_{\text{xtol}}$ with residual still above τ_{stag} .
IterationCap	"max_iters_stagnation"	Iteration budget exhausted with residual still above τ_{stag} .
LinearSolveNaN	"linear_solve_nan"	Linear-solve exception or NaN in $\delta\mathbf{x}$.
InProgress	""	Sentinel for “no break this iteration”; never visible to the caller.